

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



Práctica 3:

FilmStars

PONDERACIÓN: 3 pts

Índice

Descripción del problema	3
Alcance del sistema	4
1. Módulo de Autenticación y Sesión (Nuevo)	4
2. Selección de Ubicación y Funciones	4
3. Visualización de películas por categoría	4
4. Selección de Asientos en Tiempo real	4
5. Flujo de Compra de Boleto	4
Requisitos y restricciones	5
Requisitos Técnicos y de Arquitectura	5
1. Principios SOLID (Calidad de código)	5
2. Uso de contenedores (DevOps)	6
Entregables Requeridos	6
1. Código Fuente e Implementación	6
2. Actualización de Diseño de Software	6
3. Diseño de Software (Documentación SOLID)	7
4. Toma y justificación de Decisiones	7
5. Sección de Despliegue con Contenedores:	7
Herramientas permitidas	7
Cronograma	8
Consideraciones	8

Descripción del problema

Una importante cadena de cines desea modernizar su plataforma digital para permitir la venta de boletos en línea. El sistema debe de ser capaz de gestionar picos altos de demanda de manera eficiente garantizando que no ocurran condiciones de carrera al seleccionar asientos de forma simultánea, y evitando la pérdida de transacciones financieras ante caídas del sistema.

Para cumplir con estos atributos de calidad, el departamento de ingeniería ha determinado que el sistema se construirá bajo un enfoque de Arquitectura Orientada a Servicios (SOA). La comunicación crítica entre servicios debe gestionarse obligatoriamente de forma asíncrona, utilizando un middleware de mensajería basado en colas.

En la práctica anterior, se diseñó la arquitectura base para el sistema de reserva y venta de boletos de cine bajo un enfoque Orientado a Servicios (SOA). Para esta segunda iteración, el objetivo principal es trasladar el diseño a un sistema de software completamente funcional, elevando los estándares de calidad en la construcción del código e implementando el esquema de seguridad para el control de acceso.

El desarrollo de esta fase exige la aplicación rigurosa de los Principios SOLID de diseño orientado a objetos. Esto garantizará que los servicios (como la cartelera, usuarios o pagos) sean tolerantes al cambio, fáciles de mantener y probar de forma aislada. Asimismo, se debe asegurar la frontera del sistema implementando autenticación basada en claims mediante JWT (JSON Web Tokens).

En las iteraciones anteriores, se diseñó la arquitectura base SOA y se construyó el flujo central de cara al cliente (comprar boletos y autenticación) utilizando contenedores Docker. No obstante, una plataforma comercial requiere de un gobierno operativo robusto y de un flujo de entrega de código que garantice la estabilidad del sistema ante cambios continuos en producción.

Alcance del sistema

El sistema final entregado debe fusionar las capacidades de la práctica anterior con los nuevos componentes de seguridad:

1. Módulo de Autenticación y Sesión

- **Registro de usuarios:** Formulario con validaciones en el frontend y persistencia segura (contraseñas encriptadas en la base de datos).
- **Inicio de Sesión:** Validación de credenciales en el servicio de usuarios.
- **Emisión y Gestión de JWT:** Tras un login exitoso, el backend debe firmar y retornar un token JWT que almacene los claims esenciales del usuario (ej. ID, Nombre, Rol). El frontend debe capturar y enviar este token en la cabecera HTTP (*Authorization: Bearer <token>*) para acceder a las rutas protegidas (como el flujo de compra y selección de asientos).

2. Selección de Ubicación y Funciones

- El usuario debe de poder seleccionar su ciudad para visualizar dinámicamente los cines y los horarios/funciones disponibles en dicha localidad.

3. Visualización de películas por categoría

- Clasificación y despliegue de la cartelera segmentada según el tipo de proyección: Estrenos, Pre-ventas, Re-Estrenos.

4. Selección de Asientos en Tiempo real

- Mapa interactivo de la sala para la función seleccionada. Se debe mapear visualmente el estado de cada asiento (Disponible, Ocupado, Seleccionado/Bloqueado temporalmente)

5. Flujo de Compra de Boleto

- Procesamiento del flujo desde que se confirma la selección de asientos, se envía la cola de mensajería para su validación, se procesa el pago simulado y se emite el boleto final.

6. Módulo Administrativo (NUEVO)

El sistema debe incorporar un **Panel de Administración** web exclusivo para usuarios con rol de administrador (*Rol: Admin*), el cual debe permitir operaciones CRUD (*Crear, Leer, Actualizar, Borrar*) sobre las siguientes entidades:

- **Gestión de Cines y Salas:** Configuración de sedes físicas, número de salas y su respectivo mapa base de asientos.

- **Gestión de Horarios y Funciones:** Asignación de películas a salas específicas en horarios determinados, definiendo si la función corresponde a un *Estreno*, *Pre-venta* o *Re-estreno*.

7. Restricciones de Acceso End-to-End (NUEVO)

- Tanto las rutas del Frontend como los endpoints del Backend SOA dedicados a la administración deben validar de forma estricta que el **JWT** de la sesión pertenezca a un administrador legítimo. Queda prohibido permitir que un usuario común interactúe con estas vistas o APIs.

Requisitos de Automatización (NUEVO)

1. Estrategia de Testing (Aseguramiento de Calidad)

- **Pruebas Unitarias:** Se exige el desarrollo de pruebas unitarias automatizadas en el backend para validar de manera aislada la lógica de negocio (**por ejemplo:** reglas de negocio para evitar reservas duplicadas, cálculo de precios por tipo de función, o validación de tokens).
- **Métrica de Cobertura (Coverage):** El conjunto de pruebas unitarias debe alcanzar de forma obligatoria un mínimo del 75% de cobertura de código (*Code Coverage*) en los componentes principales de lógica de negocio del backend. Los reportes de cobertura deben ser legibles y generados por herramientas nativas del ecosistema (ej. *Jest/Mocha* para TS, *PyTest* para Python, *Go Test* con *-coverprofile* para Go)

2. CI/CD y Despliegue Multi-Entorno

Se debe configurar un pipeline de CI/CD utilizando herramientas como **GitHub Actions**, el cual constará de las siguientes fases automatizadas ante cada evento de *Merge/Pull Request* a la rama de producción:

- **Fase de integración (CI)**
 - Descarga de dependencia y compilación del proyecto
 - Ejecución automatizada de la suite de pruebas unitarias
 - **Control de calidad:** Si la cobertura es menor al 75% o alguna prueba falla, el pipeline debe detenerse inmediatamente e impedir el despliegue.
- **Fase de Despliegue (CD) - Multi-Entorno:**
 - **Entorno de Producción en la Nube (AWS):** Si la fase de CI es exitosa, el pipeline debe empaquetar automáticamente las imágenes de Docker, subirlas a un registro (Dockerhub) y desplegarlas en AWS (utilizando servicios como AWS EC2, ECS, o mediante una arquitectura orquestada por Docker Compose en la nube). La aplicación en AWS debe ser 100% accesible mediante una URL pública.
 - **Entorno de Réplica Local:** El repositorio debe mantener actualizado el archivo *docker-compose.yml* para que cualquier evaluador o desarrollador pueda replicar exactamente el entorno productivo de AWS de manera local en su máquina con un solo comando.

Requisitos y restricciones

Cada dominio conceptual (Usuario, Películas/Cartelera, Reservas/Asientos, Pagos) debe de modelarse como servicio independiente con su propio almacenamiento o lógica de acceso a datos.

Los procesos donde la concurrencia sea crítica deben de desacoplarse mediante un bróker de mensajería. Las peticiones no deben bloquear el hilo principal de la aplicación, en su lugar, deben de entrar a cola y serán procesadas por consumidores independientes.

Esta práctica **NO** consiste únicamente en la entrega de endpoints o APIs en el backend. Se requiere obligatoriamente una **aplicación web funcional de extremo a extremo (End-to-End)**, lo que incluye una interfaz de usuario (Frontend) interactiva e integrada con los servicios mediante consumo de APIs

Requisitos Técnicos y de Arquitectura

1. Principios SOLID (Calidad de código)

Los estudiantes deberán demostrar de forma explícita en su código fuente la aplicación de los siguientes principios:

- **S - Single Responsibility Principle (Responsabilidad Única)**
Cada clase, controlador o servicio de backend debe tener una única razón para cambiar (ej. separar la lógica de negocio de la reserva de la lógica de persistencia de datos o del envío a la cola de mensajería).
- **O - Open/Closed Principle (Abierto/Cerrado):**
El software debe estar abierto para su extensión pero cerrado para su modificación (ej. el sistema de cálculo de precios según el tipo de función—estreno, preventa o re-estreno—debe permitir añadir nuevas reglas sin modificar las clases existentes).
- **L - Liskov Substitution Principle (Sustitución de Liskov):**
Las clases derivadas deben poder sustituir a sus clases base sin alterar el comportamiento del programa.
- **I - Interface Segregation Principle (Segregación de Interfaces):**
Los clientes no deben ser obligados a depender de interfaces o métodos que no utilizan.
- **D - Dependency Inversion Principle (Inversión de Dependencias):**
Los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones (uso estricto de interfaces para desacoplar controladores de servicios y servicios de repositorios de datos).

2. Uso de contenedores (DevOps)

- **Dockerfiles:** Cada componente independiente de la arquitectura (Frontend web, cada uno de los servicios del Backend SOA, y las bases de datos si

requieren inicialización personalizada) debe poseer su propio archivo *Dockerfile* optimizado.

- **Docker Compose:** Se exige la creación de un archivo ***docker-compose.yml*** en la raíz del proyecto que permita orquestar e inicializar la topología completa del sistema con un único comando (*docker compose up --build*). Este archivo debe empaquetar y levantar:
 - Contenedor del Frontend
 - Contenedores de Servicios Backend SOA
 - Broker de mensajería configurado con sus respectivas colas
 - Instancias de las Bases de Datos correspondientes

Entregables Requeridos

1. Código Fuente e Implementación

- **Repositorio de Código (GitHub/GitLab):** Proyecto organizado de manera limpia con un archivo ***.md*** explicativo que incluya los pasos detallados para levantar tanto el Frontend como el Backend y el entorno con Docker Compose
- **Archivos de Configuración Docker:** Inclusión de todos los *Dockerfiles* y el script *docker-compose.yml* completamente funcional.
- **Scripts de CI/CD:** Archivo de configuración del pipeline.
- **Evidencia de Cobertura:** Captura de pantalla o logs del pipeline demostrando el cumplimiento del 75% de cobertura.
- **Aplicación End-to-End:** Frontend funcional integrado y consumiendo los servicios del backend a través de la red interna de Docker.
- Creación del Tag con la versión **V2.0.0**

2. Actualización de Diseño de Software

- Actualización de los **RF y RNF** para incluir controles de roles y CRUD de Administrador.
- Modificación de los **Casos de Uso Expandidos** incorporando los elementos del Administrador.
- **Diagrama de Secuencia:** Elementos de interacción en el panel de administrador.
- **Diagrama de actividades:** Elementos de interacción en el panel de administrador.
- Actualización de los **componentes y/o paquetes** para incluir los elementos de DevOps.
- Actualización Modelo 4+1 incluyendo lo anteriormente mencionado.

3. Diseño de Software (Documentación SOLID) - Administrador

Por cada uno de los 5 principios, el grupo deberá detallar formalmente:

- **Dónde se aplicó:** Ruta exacta del archivo, clases, componentes o interfaces. (Incluir capturas).
- **Cómo se aplicó:** Explicación técnica del patrón o estructura utilizada.
- **Por qué se aplicó:** Justificación de la mejora en mantenibilidad, acoplamiento o cohesión

4. Toma y justificación de Decisiones

Se deben de responder las preguntas de ¿Qué?, ¿por qué? y ¿para qué? para:

- Herramienta CI/CD
- Herramienta para pruebas unitarias.

5. Aseguramiento de Calidad (Testing)

- Explicación de la estrategia de pruebas unitarias adoptada.
- Rutas de los archivos de pruebas en el repositorio y justificación de los escenarios probados para alcanzar el 75% de cobertura.

6. Sección de DevOps y Despliegue Continuo

- **Diagrama de Arquitectura (Alto nivel/Bloques):** Diagrama conceptual que ilustre el flujo desde el commit en GIT, el disparo del Webhook, la ejecución de pruebas y el despliegue (De todos los elementos) en infraestructura de AWS
- **Diagrama de flujo:** Diagrama que ilustre el flujo desde el commit en GIT, el disparo del Webhook, la ejecución de pruebas y el despliegue (De todos los elementos) en infraestructura de AWS
- **Actualización del Diagrama de Despliegue:** Modificación del diagrama físico para reflejar la topología exacta desplegada en AWS (identificando las instancias virtuales, mapeo de puertos de red, volúmenes de datos del Message Broker y bases de datos en la nube).

7. Sección de Despliegue con Contenedores:

- Actualización del **Diagrama de Despliegue (UML)** para reflejar el mapeo físico de los contenedores Docker, la asignación de puertos, volúmenes de datos y la red virtualizada de Docker Compose.

Herramientas permitidas

Tipo	Categoría	Descripción
Opcional	Lenguajes	Go, TypeScript, Python
Opcional	Framework	FastAPI, Flask, Python, Express, NestJS, Gin
Opcional	Comunicación	RabbitMQ, Kafka
Opcional	Seguridad y Sesiones	JWT, Session Cookies, OAuth
Opcional	Base de datos	MSSQL, MySQL, PostgreSQL, MongoDB
Obligatorio	Control de Versiones	Github
Obligatorio	Nube	AWS
Opcional	CI/CD	Jenkins, GithubActions
Opcional	Testing	Jest/Mocha, PyTest, go test -coverprofile
Opcional	Documentación, Diseño, Modelado	Excalidraw, Figma, Canva, LucidChart, Draw.io , StarUML, Visual Paradigm, FossFlow

Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	09/06/2026	12/06/2026
Elaboración	09/06/2026	12/06/2026
Calificación	12/06/2026	11/06/2026

Consideraciones

- **Fecha límite de entrega:** 012 de Junio a las 9:00 **AM**
- **Nombre del repositorio:** SA_PRACTICA_GX
- **Colaborador:** *Samashoas*
- **Medio de entrega:** UEDI
- **Formato de entrega:** Formato Markdown que incluya tabla de integrantes, índice, introducción, desarrollo de todos los diagramas/documentación técnica y conclusiones.
- No se permite el uso de herramientas como supabase o prisma para la gestión de la base de datos (**Si está permitido el uso de otro ORM**)
- Se deben cargar los archivos crudos de la documentación al repositorio, de lo contrario el diagrama no será válido.
- Para esta fase se estará trabajando de forma local, no es necesario hacer despliegues en ninguna nube aún.
- **No se permiten commits fuera del horario establecido**